

내 손으로 조작하며 배우는 5일 강화학습

260205 4일 차: DQN과 LunarLander

4일차 학습 목표

- Q-테이블 기반 방법의 한계를 이해하고 **DQN**의 필요성을 정리
- Q-Network를 정의하고 학습 가능한 형태의 Q-function을 구현
- DQN 학습 안정화를 위한 핵심 요소를 이해
- ReplayBuffer와 전체 학습 루프를 직접 구현하여 동작 과정을 체험
- Gradio으로 DQN 학습 및 성능을 시각화하는 대시보드를 제작

Q-테이블의 한계

1. 상태 공간 폭발: 연속값이나, 차원이 커지면

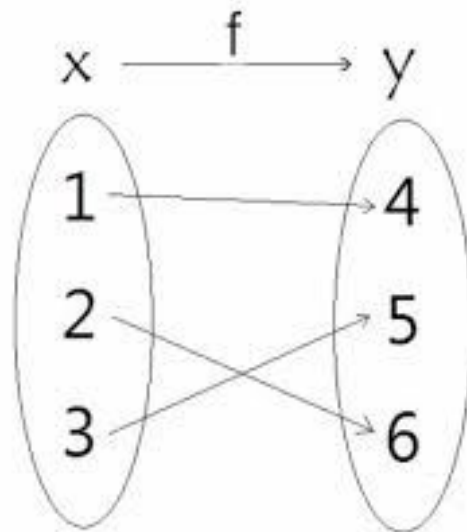
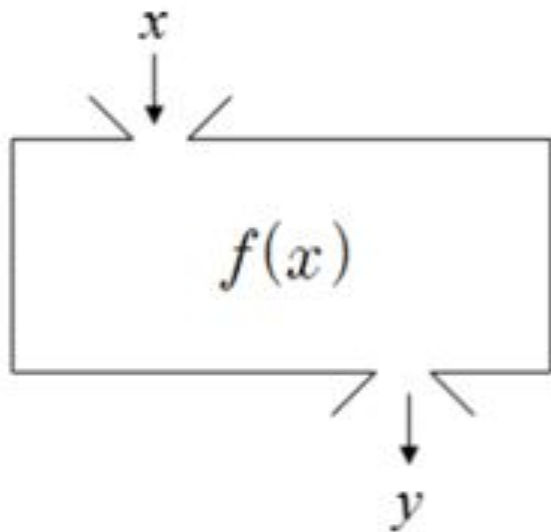
테이블 크기가 기하급수적으로 증가한다.

2. 메모리 비효율: 사용하지 않는 상태-행동 쌍에 대해서도 값을 저장

3. 일반화 불가: 본 적 없는 새로운 상태에 대해 **Q** 값을 추정 못 함

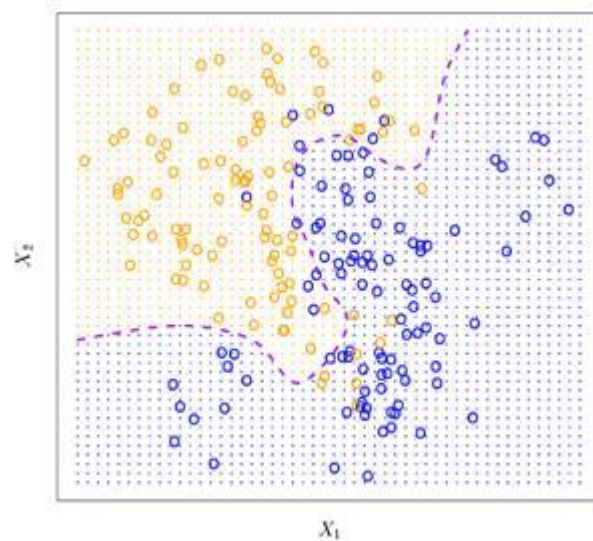
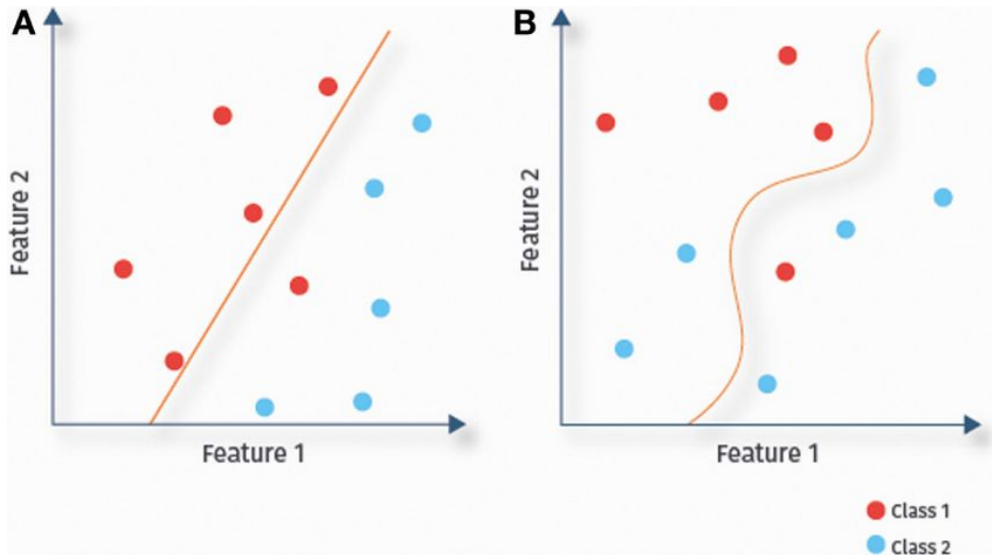
Q-테이블의 한계

- $Q(s,a) \rightarrow$ 특정 값



신경망(Neural Network)

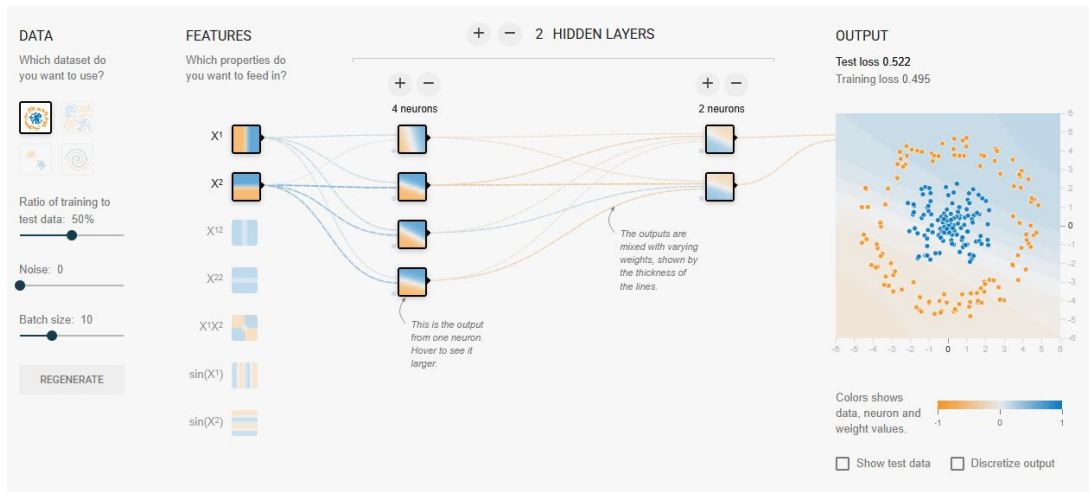
- Universal Approximation Capability (범용 근사능)
 - 은닉층이 하나 있는 신경망은 충분한 뉴런이 있으면 모든 연속함수를 근사할 수 있다



신경망(Neural Network)

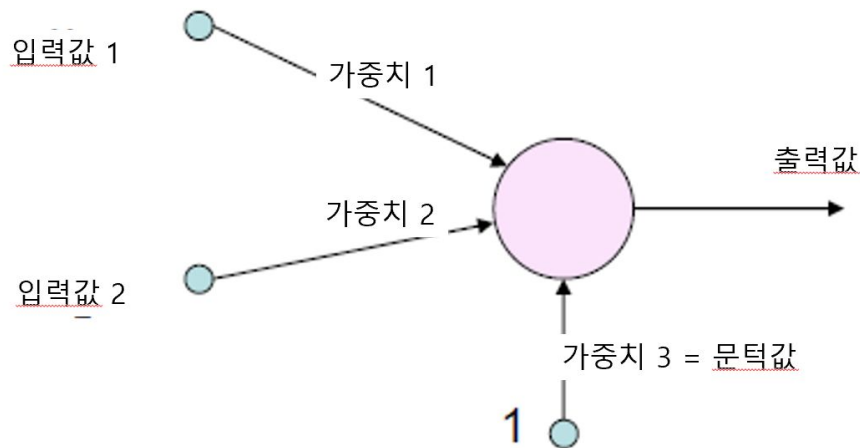
- Universal Approximation Capability (범용 근사능)
 - 은닉층이 하나 있는 신경망은 충분한 뉴런이 있으면 모든 연속함수를 근사할 수 있다

<http://playground.tensorflow.org>



퍼셉트론(Perceptron)

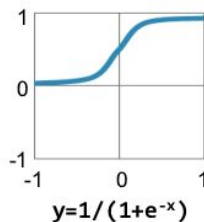
- 생물학적 뉴런의 기능을 모방
- 다수의 입력 신호를 받아 하나의 출력 신호를 생성



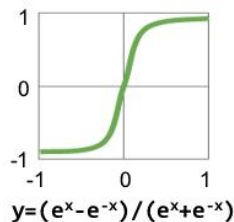
활성 함수 (Activation Function)

Traditional
Non-Linear
Activation
Functions

Sigmoid

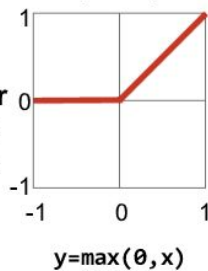


Hyperbolic Tangent

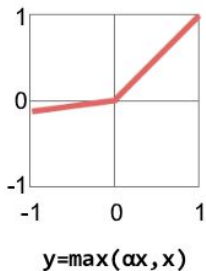


Modern
Non-Linear
Activation
Functions

Rectified Linear Unit
(ReLU)

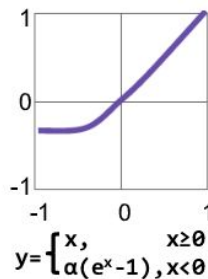


Leaky ReLU



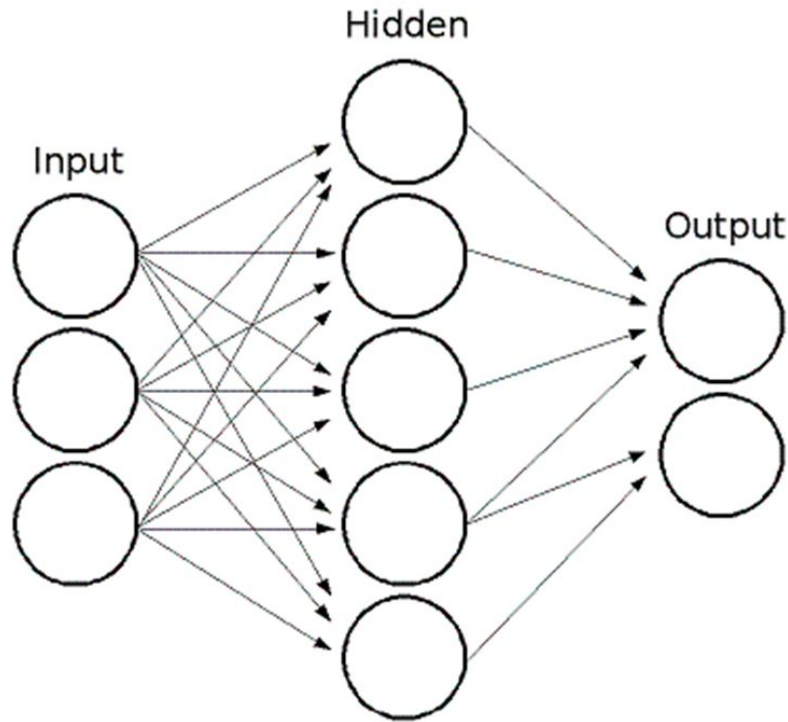
$\alpha = \text{small const. (e.g. 0.1)}$

Exponential LU



인공신경망(Artificial Neural Network)

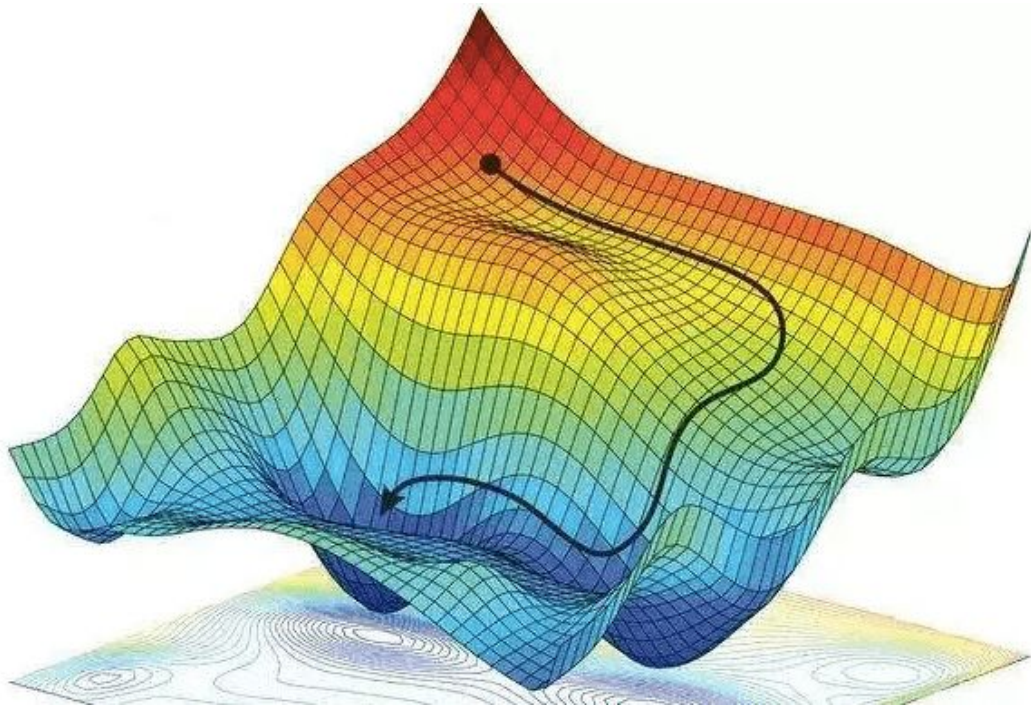
- 다층 퍼셉트론(Multi-Layer Perceptron)
- 인간의 두뇌 구조를 모방
- 입력층, 숨겨진 층, 출력층
- 비선형 관계를 모델링
- 층이 4개 이상이 되면 딥러닝



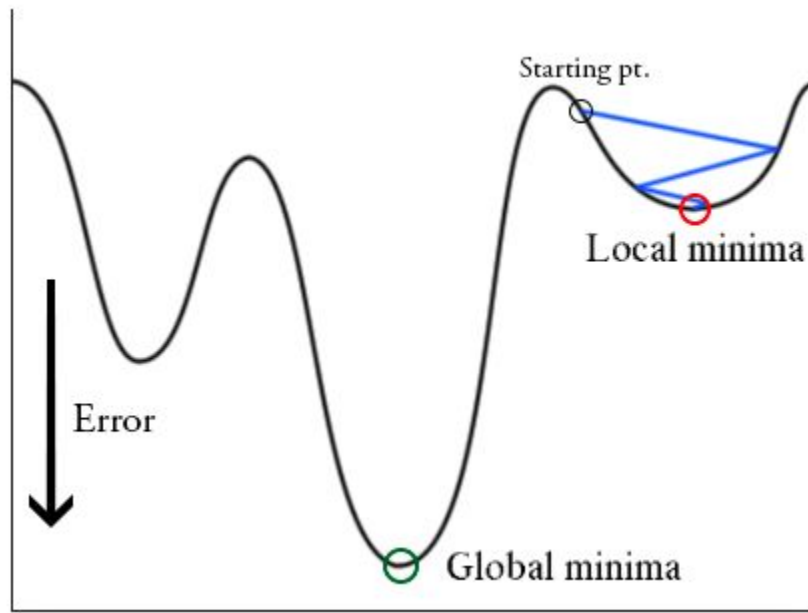
역전파(Backpropagation)

- 출력이 목표 출력과 얼마나 다른지를 오차를 계산
- 오차를 출력에서 입력쪽으로 역으로 전파하여 가중치를 조정
- 경사 하강법(Gradient Descent)
 - 신경망의 손실 함수(Loss Function)를 최소화

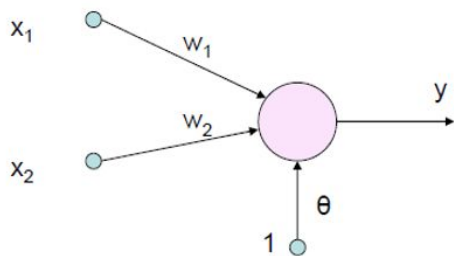
경사 하강법(Gradient Descent)



경사 하강법(Gradient Descent)



역전파(Backpropagation)



X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

Other Initial Values

$W1$: 0.7

$W2$: 0.3

Theta: 0.5

Learning rate: 0.2

x1	x2	$X1*w1$	$X2*w2$	Sum	Theta	Y	Expected Y	Same?
0	0	0	0	0	0.5	0	0	Yes
0	1	0	0.3	0.3	0.5	0	0	Yes
1	0	0.7	0	0.7	0.5	1	0	No

역전파(Backpropagation)

Error: (Expected Y) - Y $\rightarrow 0 - 1 = -1$

Delta: Learning rate*Error $\rightarrow -1*0.2 = -0.2$

W1_delta = w1*delta, W2_delta = w2*delta

New_W1: $0.7 - 0.2 = 0.5$, New_W2: 0.3

Other Initial Values

W1: 0.7

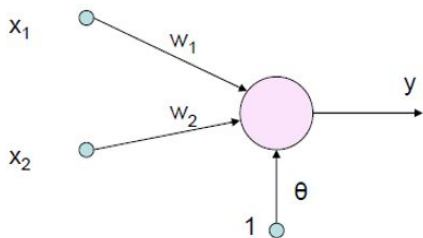
W2: 0.3

Theta: 0.5

Learning rate: 0.2

x1	x2	X1*w1	X2*w2	Sum	Theta	Y	Expected Y	Same?
0	0	0	0	0	0.5	0	0	Yes
0	1	0	0.3	0.3	0.5	0	0	Yes
1	0	0.7	0	0.7	0.5	1	0	No

역전파(Backpropagation)



X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

Other Initial Values

$W1$: 0.5

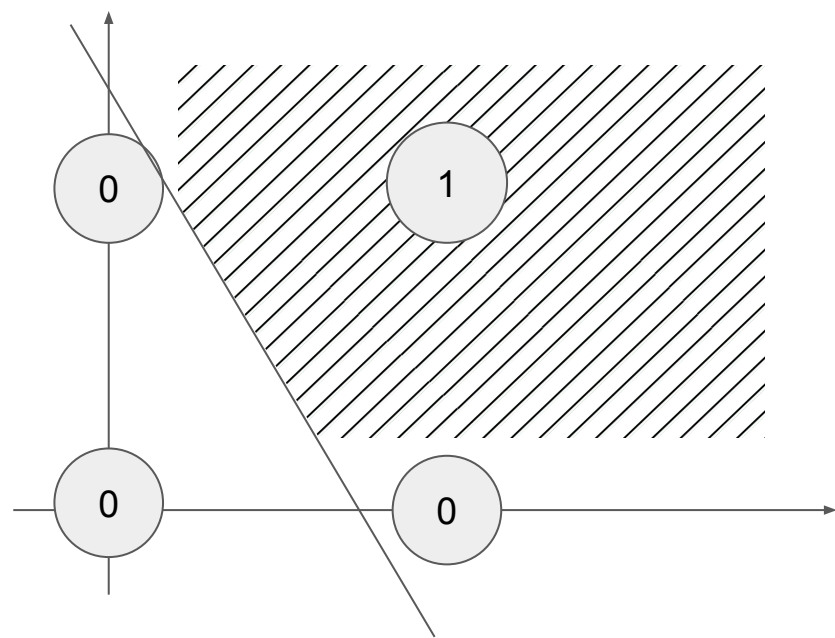
$W2$: 0.3

Theta: 0.5

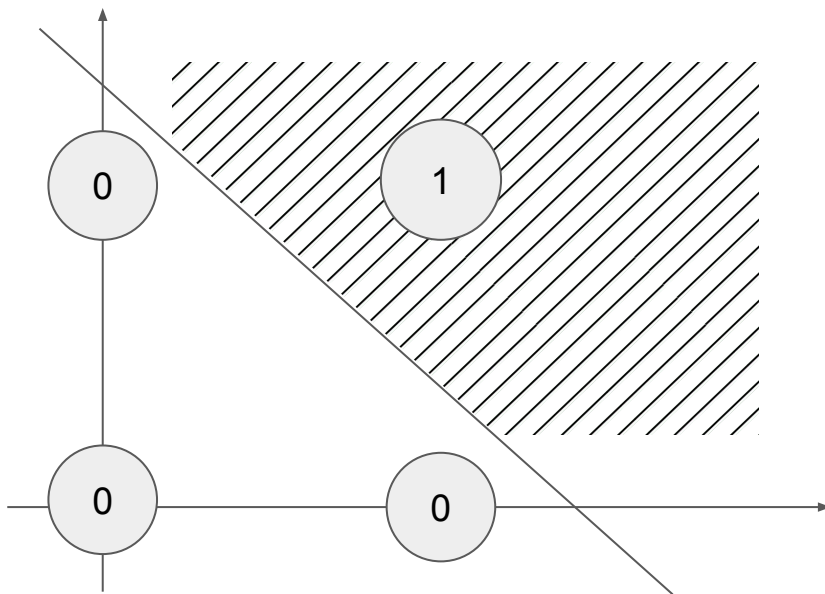
Learning rate: 0.2

x1	x2	$X1*w1$	$X2*w2$	Sum	Theta	Y	Expected Y	Same?
0	0	0	0	0	0.5	0	0	Yes
0	1	0	0.3	0.3	0.5	0	0	Yes
1	0	0.5	0	0.5	0.5	0	0	Yes
1	1	0.5	0.3	0.8	0.5	1	1	Yes

역전파(Backpropagation)



역전파(Backpropagation)



모델 성능 평가 지표들

- 정확도 (Accuracy): 전체 예측 중에서 올바르게 예측한 비율
- 정밀도 (Precision): 모델이 양성으로 예측한 것 중 실제로 양성인 비율.
- 재현율 (Recall): 실제 양성인 샘플 중에서 모델이 양성으로 올바르게 예측한 비율
- F1-점수 (F1 Score):
정밀도와 재현율의 조화 평균. 두 지표 간의 균형을 평가.
데이터셋 클래스 불균형이 있는 상황에서 유용.

모델 성능 평가 지표들

- Type 1 Error: 병에 걸리지 않은 사람을 병에 걸렸다고 오진
- Type 2 Error: 병에 걸린 사람을 병에 걸리지 않았다고 오진

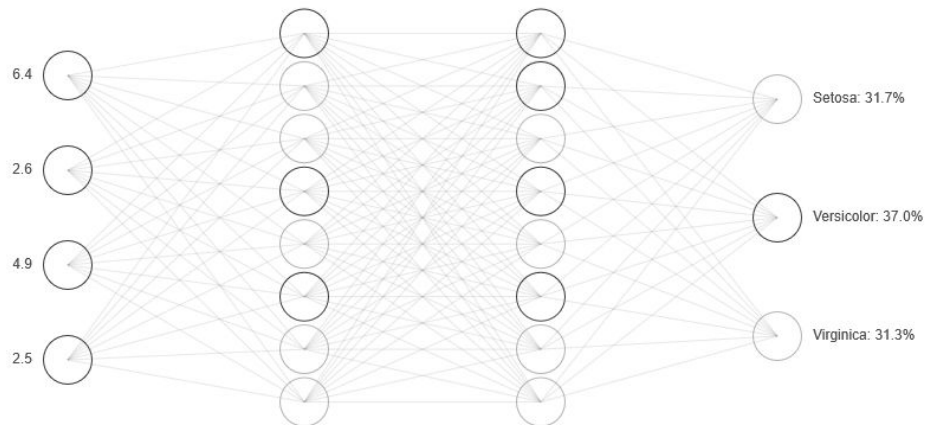
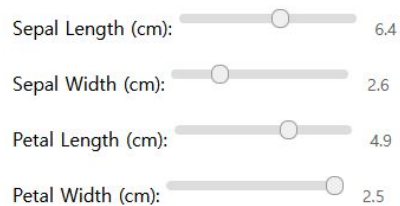
	Actual -- True/False	
	True	False
Predicted -- Positive/Negative	True Positive	False Positive (Type I)
	False Negative (Type II)	True Negative

모델 성능 평가 지표들

		predicted condition			
total population		prediction positive	prediction negative	Prevalence = $\frac{\Sigma \text{ condition positive}}{\Sigma \text{ total population}}$	
true condition	condition positive	True Positive (TP)	False Negative (FN) (type II error)	True Positive Rate (TPR), Sensitivity, Recall, Probability of Detection $= \frac{\Sigma \text{ TP}}{\Sigma \text{ condition positive}}$	False Negative Rate (FNR), Miss Rate $= \frac{\Sigma \text{ FN}}{\Sigma \text{ condition positive}}$
	condition negative	False Positive (FP) (Type I error)	True Negative (TN)	False Positive Rate (FPR), Fall-out, Probability of False Alarm $= \frac{\Sigma \text{ FP}}{\Sigma \text{ condition negative}}$	True Negative Rate (TNR), Specificity (SPC) $= \frac{\Sigma \text{ TN}}{\Sigma \text{ condition negative}}$
Accuracy $= \frac{\Sigma \text{ TP} + \Sigma \text{ TN}}{\Sigma \text{ total population}}$		Positive Predictive Value (PPV), Precision $= \frac{\Sigma \text{ TP}}{\Sigma \text{ prediction positive}}$	False Omission Rate (FOR) $= \frac{\Sigma \text{ FN}}{\Sigma \text{ prediction negative}}$	Positive Likelihood Ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic Odds Ratio (DOR) $= \frac{\text{LR+}}{\text{LR-}}$
		False Discovery Rate (FDR) $= \frac{\Sigma \text{ FP}}{\Sigma \text{ prediction positive}}$	Negative Predictive Value (NPV) $= \frac{\Sigma \text{ TN}}{\Sigma \text{ prediction negative}}$	Negative Likelihood Ratio (LR-) = $\frac{\text{FNR}}{\text{TNR}}$	

인공신경망의 시각화

day4/01_iris_visualization.html



인공신경망의 시각화 자료

https://adamharley.com/nn_vis/mlp/2d.html

- MNIST (필기 숫자 인식) 학습에 대한 시각화:

Deep Q-Network (DQN)

- 2015년 구글 딥마인드에서 발표
- 딥러닝 + Q-Learning
- 경험 재생(Experience Replay)
- 타겟 네트워크(Target Network)

Deep Q-Network (DQN) vs Q-러닝

- 보상 표현 방식
 - DQN은 신경망을 사용하여 Q-함수를 근사화하여 가치를 추정
- 메모리와 확장성
 - DQN은 신경망을 통해 연속적이거나 아주 큰 상태/행동 공간 처리 가능

순수 DQN의 약점

- 샘플 간 강한 상관관계 (Correlation): 연속된 경험을 그대로 학습.
- 환경·초기값에 따라 학습 경로가 크게 달라지는 민감성.



순수 DQN의 약점

- 타깃 값도 학습 중인 네트워크에 의존 → Bootstrapping 불안정.
- Q 값을 과도하게 크게 추정하는 overestimation 문제.



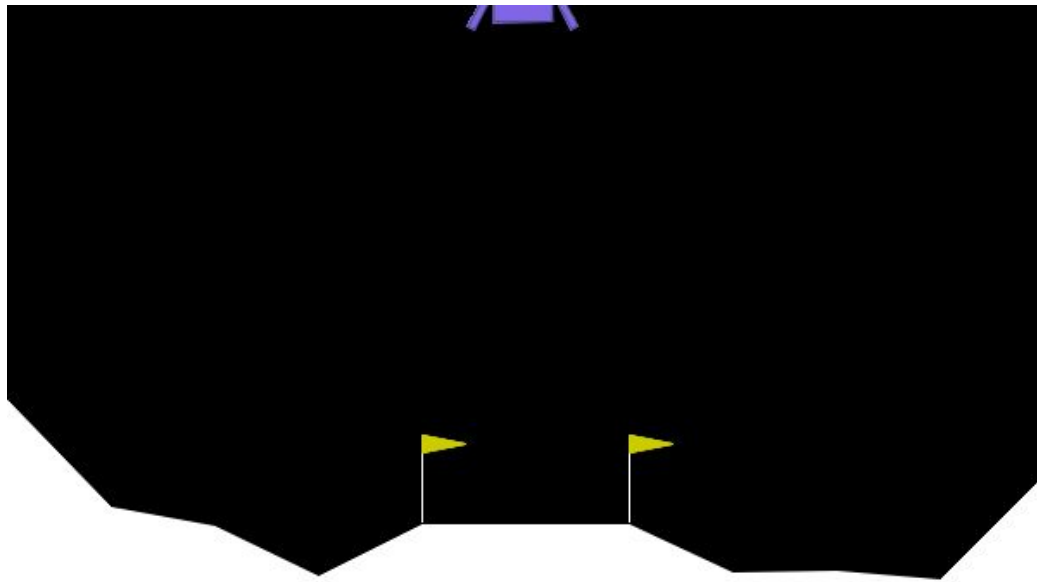
Deep Q-Network (DQN)

- Experience Replay
 - 경험을 버퍼에 모아두고, 무작위로 샘플링하여 학습한다.
- Target Network
 - Q-네트워크를 2개 운용하여 하나는 학습하는데 사용하고, 다른 하나는 행동을 예측하는데 사용

```
python day4/02_cartpole_compare.py
```

LunarLander-v3

day4/03_dqn_lunarlander_colab.ipynb



LunarLander-v3 - 상태

- X 좌표: $-1.5 \sim 1.5$. 화면 중앙이 0. 양수면 오른쪽, 음수면 왼쪽. 착륙 패드는 (0,0)에 위치.
- Y 좌표: $-1.5 \sim 1.5$. 화면 상단이 1.5, 착륙 패드 바닥이 0.
- X 속도: $-5 \sim 5$. 수평 이동 속도.
- Y 속도: $-5 \sim 5$. 수직 이동 속도 (음수면 하강 중).
- 각도 (Angle): $-3.14 \sim 3.14$ (라디안). 착륙선의 기울기. 0이면 수직으로 똑바로 서 있는 상태.
- 각속도: $-5 \sim 5$. 회전하는 속도.
- 왼쪽 다리 접촉: 0 또는 1. 왼쪽 다리가 땅에 닿았으면 1, 아니면 0.
- 오른쪽 다리 접촉: 0 또는 1. 오른쪽 다리가 땅에 닿았으면 1, 아니면 0.

LunarLander-v3 - 행동

- 4개의 이산적(Discrete) 행동 공간
 - 0: Do nothing (아무것도 안 함, 자유 낙하)
 - 1: Fire left engine (왼쪽 엔진 점화 → 오른쪽으로 힘을 받음)
 - 2: Fire main engine (메인 엔진 점화 → 위쪽으로 힘을 받음, 중력 상쇄)
 - 3: Fire right engine (오른쪽 엔진 점화 → 왼쪽으로 힘을 받음)
- 참고: continuous=True 옵션을 주면
메인 엔진 힘(-1~1)과 측면 엔진 힘(-1~1)을 연속적인 값으로 변경

LunarLander-v3 - 보상

- 거리 보상: 착륙 패드(0,0)에 가까워질수록 점수를 받음.
- 속도 보상: 속도가 0에 가까워질수록(천천히 움직일수록) 점수를 받음.
- 기울기 보상: 기체가 수평(각도 0)에 가까울수록 점수를 받음.
- 다리 접촉: 다리가 땅에 닿으면 각각 +10점 추가.
- 연료 소비 페널티:
 - 메인 엔진 사용 시: 매 프레임 -0.3점 (최대한 아껴 써야 함).
 - 측면 엔진 사용 시: 매 프레임 -0.03점.
- 종료 보상 (큰 점수):
 - 안전 착륙 (Rest): 기체가 멈추면(속도 ≈ 0) +100점 (성공).
 - 추락 (Crash): 기체가 부서지면 -100점 (실패).

LunarLander-v3 - 종료 조건

- 추락 (Crashed): 기체가 땅에 너무 세게 부딪히거나, 몸통이 땅에 닿음.
- 정지 (Landed): 기체가 안전하게 착륙 패드 위에 멈춤 (Sleep 상태).
- 화면 이탈: 기체의 X 좌표가 ± 1.0 을 넘어가 화면 밖으로 나감. (단, 위쪽은 관촬음)
- 시간 초과 (Truncated): 정해진 최대 스텝(보통 400~1000스텝)을 넘길 경우 강제 종료.

Stable Baselines3

- OpenAI Gym과 PyTorch를 기반
- Scikit-Learn과 비슷하게 강화학습 알고리즘 모음
- 강화학습을 쉽게 적용하고 테스트 가능.

Deep Q-Network (DQN)

1. 환경에서 상태 관찰
2. 행동 선택 (ϵ -greedy 정책에 의해 탐험과 경험 활용의 분배)
3. 보상 획득 및 다음 상태 저장
4. 경험 재플레이에 의한 신경망 업데이트
5. 타깃 네트워크 주기적 업데이트

```
python day4/04_policy_kwargs_dqn.py
```

```
python day4/05_dqn_param_effect.py
```

```
day4/06_callback_reward_loss_gradio.ipynb
```

Deep Q-Network (DQN) - 하이퍼 파라미터

- learning_rate
- buffer_size
- learning_starts
- batch_size
- gamma
- tau
- target_update_interval
- train_freq
- gradient_steps
- exploration_fraction
- exploration_initial/final_eps
- policy_kwargs (net_arch)
- policy_kwargs (activation_fn)
- max_grad_norm
- seed

REINFORCE 알고리즘 정책

- REward Increment = Nonnegative Factor × Offset Reinforcement × Characteristic Eligibility
- On-Policy 강화학습
- 이기면 그 수(행동)의 확률을 높이고, 지면 확률을 낮춘다.
 - 몬테카를로 정책 경사(Monte Carlo Policy Gradient)
- 에피소드가 끝날 때까지 기다렸다가 학습
- 분산(Variance)이 큰 단점

`day4/07_reinforce_gradio_mountaincar.ipynb`

On-Policy vs Off-Policy

Off-Policy (값 기반)

- 대표: Q-Learning, DQN
- 특징: 과거 경험도 학습 가능 (데이터 효율 ↑)
- 방식: Q-Table/Network (간접 선택)

On-Policy (정책 기반)

- 대표: REINFORCE, PPO
- 특징: 현재 정책 경험만 학습 (안정성 ↑)
- 방식: Policy Network (직접 선택)

`python day4/08_onpolicy_compare.py`

머신러닝 추천 강의

인프런 - 나도코딩 파이썬 무료강의 활용편7 머신러닝

- 파이썬 문법, 알고리즘 보다 머신러닝 원리, 실제 예제를 통한 실무 위주 강의

AWS - Basics of Machine Learning

- 아마존 AWS 에서 제공하는 머신러닝 강의
- AWS 제품 사용을 익히면서 머신러닝 학습

머신러닝 추천 강의

네이버 부스트 코스 - Andrew Ng 교수님 딥러닝 코스

- 네이버 아이디 필요, 한글 자막, 자료 제공



딥러닝 1단계: 신경망과 딥러닝

Andrew Ng | 부스트코스

♥ 1251 👤 11054 📺 기본

무료강의



딥러닝 2단계: 심층 신경망 성능 향상시키기

Andrew Ng | 부스트코스

♥ 244 👤 3576 📺 기본

무료강의



딥러닝 3단계: 머신러닝 프로젝트 구조화하기

Andrew Ng | 부스트코스

♥ 69 👤 3329 📺 기본

무료강의



딥러닝 4단계: 합성곱 신경망 네트워크 (CNN)

Andrew Ng | 부스트코스

♥ 283 👤 4319 📺 기본

무료강의

머신러닝 추천 강의

[Great Learning - Python for Machine Learning](#)

- Numpy, Pandas, Matplot 등 머신러닝 데이터 다루기, 시각화 기초부터 시작하는 강의

[coursera - Python의 응용 머신러닝, University of Michigan](#)

데이터 셋 구하기

AI Hub

- 이미지, 비디오, 오디오 등 다양한 형태의 한국형 데이터셋을 제공
- 한국어 데이터를 구하는데 유용

공공데이터 포털

- 행정안전부가 운영하는 국내 공공데이터 사이트
- Open API 형태로도 제공

데이터 셋 구하기

Kaggle

- 전 세계에서 가장 큰 데이터 사이언스 커뮤니티 플랫폼
- 다른 사용자들이 공유한 코드와 노트북로 학습 가능
- 실제 대회 문제 제공, 또는 대회 개최

ImageNet

- 대표적인 대규모 이미지 분류 데이터셋

감사합니다